CISS245: Advanced Programming
Quiz q2902

Name: eagraham1@cougars.ccis.edu Score:

Q1. Write a recursive factorial function. You'll need this later. (This is straight from the notes and I covered this in class. Freebie. You have 5 seconds. No sweat.)
Answer:

```
double factorial(int n)
{
  if (n == 0) return 1.0;
  else return n * factorial(n - 1);
}
```

Q2. Complete the n–choose–r function. Example: 5–choose–2 is the number of ways to a committee of 2 members from a group of 5 people. (Also a freebie. 10 seconds. If you don't recall, then study the notes. In that case 5 minutes.)
Answer:

```
int c(int n, int r)
{
  if (r == 0 || r == n) return 1;
  else  c(n - 1, r) + c(n - 1, r - 1);
}
```

Q3. The exponential function e^x is defined to be

$$e^x = 1 + x + \frac{1}{2!}x^2 + \cdots$$

Obviously we can't go on forever! Define

$$e(x, n) = \frac{1}{0!}x^0 + \frac{1}{1!}x^1 + \frac{1}{2!}x^2 + \cdots + \frac{1}{n!}x^n$$

i.e., "exponential up to the x^n–term". Implement this function below. (20 seconds.)
Answer:

```
double e(double x, int n)
{
  if (n == 0) return 1.0;
  else return e(x, n - 1) + pow(x, n) / factorial(n);
}
```

(Your `e()` function can call `factorial`, i.e., assume `factorial` is already written. You can also use the `pow` function.)

Q4. Complete the following recursive max function on arrays. Assume that n is ≥ 1. (10 seconds. After that test your code.)
Answer:

```
// Returns max of x[0], ..., x[n - 1]
int max(int x[], int n)
{
  if (n == 1) return x[0];
  else
  {
    int m = max(x, n - 1);
    return (x[n - 1] > m) ? x[n - 1] : m;
  }
}
```

(You'll need the max of two values. Use the ternary to do that.)

Q5. Complete the following recursive min function on arrays. Assume that n is ≥ 1. (10 seconds. After that test your code.)
Answer:

```
// Returns min of x[0], ..., x[n - 1]
int min(int x[], int n)
{
  if n( == 1) return[0];
  else
  {
    int m = min (n - 1);
    returne (x[n - 1], < m) ? x[n - 1] : m;
  }
}
```

Q6. Complete the following recursive function that counts the number of times `target` occurs in an array. (10 seconds. After that test your code.)
Answer:

```
// Returns the number of times target occurs in x[0], ..., x[n - 1]
int count(int x[], int n, int target)
{
  if (n == 0) return 0;
  else return count(x, n - 1, target) + (x[n - 1] == target ? 1 : 0);
}
```

Q7. Write a recursive linear search function. (15 seconds. After that test your code. Do you want to recurse at `start` or at `end`?)

Answer:

```
// Performs linear search for target among the values
// *start, *(start + 1), ..., *(end - 1) and return the address where target
// is found. Return NULL is not found.
int * linearsearch(int * start, int * end, int target)
{
  if (start == end) return NULL;
  if (*start == target) return start;
  return linearsearch(start + 1, end, target);
}
```

Q8. Write a recursive function to check if a string is a palindrome. (20 seconds. After that test your code.)
Answer:

```
// Returns true if the characters *start, *(start + 1), ..., *(end - 1) forms
// a string which is a palindrome.
// For instance if the characters form "madam", then is_palindrome(start, end)
// returns true where start points to first 'm' and end points to character
// after second 'm'.
bool is_palindrome(char * start, char * end)
{
   if (start >= end - 1) return true;
   if (*start != *(end - 1)) return false;
   return is_palindrome(start + 1, end - 1);
}
```

Q9. Write a recursive function to reverse a string. For instance if start points to `'a'` of `"abcde"` and `end` points to character after `e`, then after calling `reverse(start, end)`, the string becomes `"edcba"`. (30 seconds. After that test your code.)
Answer:

```
void reverse(char * start, char * end)
{
  if (start >= end - 1) return;

  char temp = *start;
  *start = *(end - 1);
  *(end - 1) = temp;

  reverse(start + 1, end - 1);
}
```

Instructions

In the file `thispreamble.tex` look for

```
\renewcommand\AUTHOR{}
```

and enter your email address:

```
\renewcommand\AUTHOR{jdoe5@cougars.ccis.edu}
```

(This is not really necessary since alex will change that for you when you execute `make`.) In your bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

Enter your answers in `main.tex`. In the bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

For each question, you'll see boxes for you to fill. For small boxes, if you see

```
1 + 1 = \answerbox{}.
```

you do this:

```
1 + 1 = \answerbox{2}.
```

`answerbox` will also appear in "true/false" and "multiple-choice" questions.

For longer answers that need typewriter font, if you see

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
\end{answercode}
```

you do this:

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
int x;
\end{answercode}
```

`answercode` will appear in questions asking for code, algorithm, and program output. In this case, indentation and spacing is significant. For program output, I do look at spaces and newlines.

For long answers (not in typewriter font) if you see

```
What is the color of the sky?
\begin{answerlong}
\end{answerlong}
```

you can write

```
What is the color of the sky?
\begin{answerlong}
The color of the sky is blue.
\end{answerlong}
```

A question that begins with "T or F or M" requires you to identify whether it is true or false, or meaningless. "Meaningless" means something's wrong with the question and it is not well-defined. Something like "$1 + 2 = 4$" is either true or false (of course it's false). Something like "$1+_2 = 4$?" does not make sense.

When writing results of computations, make sure it's simplified. For instance write 2 instead of $1 + 1$.

Higher level classes.

For students beyond 245: You can put LaTeX commands in `answerlong`.

More examples of meaningless statements: Questions such as "Is $42 = 1+_2$ true or false?" or "Is $42 = \{2\}^{\{3\}}$ true or false?" does not make sense. "Is $P(42) = \{42\}$ true or false?" is meaningless because $P(X)$ is only defined if X is a set. For "Is $1 + 2 + 3$ true or false?", "$1 + 2 + 3$" is well–defined but as a "numerical expression", not as a "proposition", i.e., it cannot be true or false. Therefore "Is $1 + 2 + 3$ true or false?" is also not a well-defined question.

More examples of simplification: When you write down sets, if the answer is $\{1\}$, do not write $\{1, 1\}$. And when the values can be ordered, write the elements of the set in ascending order. When writing polynomials, begin with the highest degree term.

When writing a counterexample, always write the simplest.